

MIZN: A spectral solver for the noiseless K=3 CRRA rational-expectations equilibrium

Companion paper to the MIZN repository

Abstract

We document the noiseless K=3 CRRA rational-expectations equilibrium (REE) problem and the architecture of MIZN, a modular solver for it. The solver is structured as the classical 5-step Hellwig cycle: conjecture price, Bayesian learning, individual demand, aggregate demand, market clearing. We make explicit how the abstract fixed-point equation $\Phi[P] = P$ factors into these five sub-equations (Sec. 4). The recommended discretization is symmetric Chebyshev expansion in atanh-stretched coordinates with a sigmoid lift; with $S_3 \times Z_2$ symmetry this reduces the problem to ~ 228 unknowns at $N = 12$, where pure (dense) Newton converges quadratically.

1 Introduction

A canonical question in financial economics is whether equilibrium prices fully aggregate the dispersed private information of traders. The Hellwig (1980) CARA–Gaussian model with supply noise admits a closed-form *partially revealing* (PR) equilibrium; the Grossman–Stiglitz (1980) paradox shows that markets cannot be too informationally efficient.

The noiseless K=3 CRRA REE studied here removes Hellwig’s supply noise (making FR equilibria more likely) but replaces CARA with CRRA on a binary asset (introducing a Jensen-gap mechanism that can support PR). Whether the PR equilibrium exists for small risk aversion γ , and what its properties are, requires a robust nonlinear fixed-point solver.

MIZN is that solver, written modularly so each of the 5 Hellwig steps lives in its own file and is independently testable. This paper documents the mathematics, the equation-by-equation decomposition of the operator, and the recommended numerical approach.

2 Economic primitives

The asset. A single risky asset has uncertain binary payoff

$$v \in \{0, 1\}, \quad \Pr(v = 0) = \Pr(v = 1) = \frac{1}{2}. \quad (1)$$

The agents. There are $K = 3$ symmetric agents indexed by $k \in \{1, 2, 3\}$. Each agent k receives one private noisy signal

$$u_k = v + \varepsilon_k, \quad \varepsilon_k \sim \mathcal{N}(0, \tau^{-1}), \quad (2)$$

with the ε_k independent given v . The signal precision τ is the same across agents.

The conditional signal density is

$$f_v(u) = \sqrt{\frac{\tau}{2\pi}} \exp\left(-\frac{\tau}{2}(u - m_v)^2\right), \quad m_0 = -\frac{1}{2}, \quad m_1 = +\frac{1}{2}, \quad (3)$$

shown in Fig. 1.

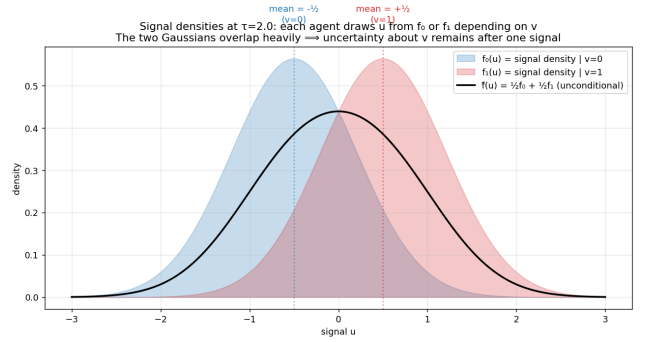


Figure 1. Conditional signal densities f_0, f_1 at $\tau = 2$. Substantial overlap means a single signal does not fully reveal v .

Preferences and demand. Each agent maximizes CRRA expected utility. Specialized to a binary asset, the demand function reads

$$x_k(\mu_k, P; \gamma) = \frac{\mu_k - P}{\gamma \mu_k (1 - \mu_k)}, \quad (4)$$

where $\mu_k = \Pr(v = 1 \mid \mathcal{I}_k)$ is agent k ’s posterior and P is the price. Demand is monotone in the wedge $\mu_k - P$ and inversely in risk aversion γ and binomial belief variance $\mu_k(1 - \mu_k)$.

Market and timing. A continuum of traders within each agent type; one shot per period; agents form posteriors and submit demands simultaneously, having access to the *conjectured* price function $P(\cdot)$; the market clears at the price that makes aggregate excess demand vanish. The rational-expectations condition is that the cleared price equals the conjectured one (Fig. 2).



Figure 2. One-period timeline for the K=3 CRRA REE.

3 The fixed-point equation

The unknown is the equilibrium price function

$$P : \mathbb{R}^3 \rightarrow [0, 1], \quad (u_1, u_2, u_3) \mapsto P(u_1, u_2, u_3). \quad (5)$$

The 5-step Hellwig cycle defines an operator Φ on price functions (Fig. 3). The REE is its fixed point:

$$\Phi[P^*] = P^*. \quad (6)$$

The 5-step Hellwig fixed-point cycle Φ — iterate until $P_{\text{new}} = P$

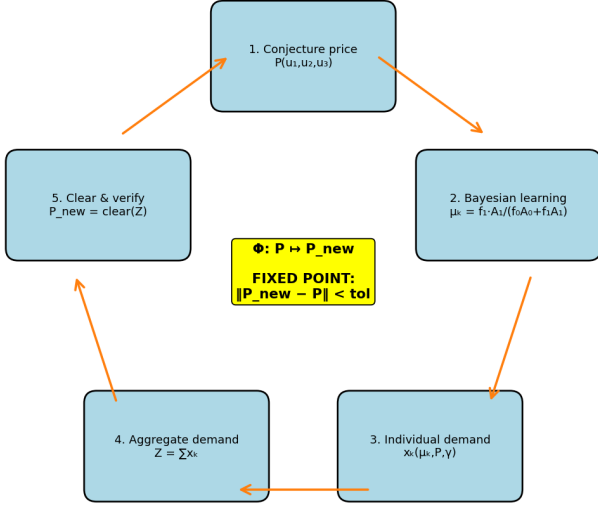


Figure 3. The 5-step Hellwig cycle, $\Phi : P \mapsto P_{\text{new}}$.

Equation (6) is one equation in an infinite-dimensional space; to make it computational we must spell out what Φ does at each cube point. *That decomposition is the topic of the next section, and it is the heart of MIZN's architecture.*

4 How Φ factors into 5 sub-equations

This section answers explicitly: what does Φ do at a single cube point (u_1, u_2, u_3) ? We factor it into the five sub-equations labelled by the 5 Hellwig steps. Each sub-equation becomes one module in the MIZN source layout.

Inputs and outputs

Input: the conjectured price function $P^{\text{conj}} : \mathbb{R}^3 \rightarrow [0, 1]$ (the current iterate), and the cube point (u_1, u_2, u_3) at which we want to evaluate $\Phi[P^{\text{conj}}]$.

Output: a single number $P_{\text{new}}(u_1, u_2, u_3) \in [0, 1]$.

4.1 Step 1 — Conjecture

The conjecture is just the input: $P^{\text{conj}} = P$. No computation. In MIZN this is the `step1_conjecture.py`

module, which only matters on the very first iteration (initial guess).

4.2 Step 2 — Bayesian learning

For each agent $k \in \{1, 2, 3\}$, compute the posterior on $v = 1$ given the agent's information $\mathcal{I}_k = \{u_k, P^{\text{conj}}\}$ AND the hypothetical realized price p (which will be the eventual market-clearing price; for now treat it as a free variable).

By Bayes,

$$\mu_k(u_k, p) = \frac{f_1(u_k) A_1(p; u_k)}{f_0(u_k) A_0(p; u_k) + f_1(u_k) A_1(p; u_k)}, \quad (7)$$

where $A_v(p; u_k)$ is the *co-area evidence* for value v :

$$A_v(p; u_k) = \int_{\{P^{\text{conj}}(u_{-k})=p\}} f_v(u_a) f_v(u_b) \frac{d\sigma}{|\nabla_{(u_a, u_b)} P^{\text{conj}}|}. \quad (8)$$

Here $u_{-k} = (u_a, u_b)$ are the two signals not seen by agent k , and the integration is over the contour where the conjectured price function equals the realized price p .

The function $\mu_k(u_k, \cdot) : [0, 1] \rightarrow [0, 1]$ is the *belief-as-a-function-of-price* curve for agent k . Computing it requires the contour integral (8), which is the numerically hardest part of the operator and is the subject of Sec. 6–8. In MIZN this lives in `step2_learning/`, with variants for kernel co-area, strict- $h = 0$ spline, and Chebyshev-spectral implementations.

4.3 Step 3 — Individual demand

Given posteriors $\mu_k(u_k, p)$ for each agent at hypothetical price p , each agent's demand is

$$x_k(u_k, p; \gamma) = \frac{\mu_k(u_k, p) - p}{\gamma \mu_k(u_k, p) (1 - \mu_k(u_k, p))}. \quad (9)$$

This is just the CRRA demand (4) evaluated at the state-dependent posterior. The price p enters *twice*: through the demand wedge $\mu_k - p$ and through the posterior $\mu_k(u_k, p)$ itself (via the price-conditioned Bayesian learning). MIZN: `step3_demand/`.

4.4 Step 4 — Aggregate demand

Sum across agents:

$$Z(u_1, u_2, u_3, p; \gamma) = \sum_{k=1}^3 x_k(u_k, p; \gamma). \quad (10)$$

MIZN: `step4_aggregate.py`.

4.5 Step 5 — Market clearing

The new price $P_{\text{new}}(u_1, u_2, u_3)$ is the value of p that makes aggregate excess demand vanish:

$$Z(u_1, u_2, u_3, P_{\text{new}}; \gamma) = 0. \quad (11)$$

This is a 1-D root-find in p . Substituting (9) into (10) and setting (11), the clearing equation spelled out is

$$\sum_{k=1}^3 \frac{\mu_k(u_k, P_{\text{new}}) - P_{\text{new}}}{\gamma \mu_k(u_k, P_{\text{new}})(1 - \mu_k(u_k, P_{\text{new}}))} = 0, \quad (12)$$

where each μ_k also depends on P_{new} via the *conjectured* price function P^{conj} entering the co-area integral (8). MIZN: `step5_clearing/` (a bisection on log-odds; see Algorithm 1).

4.6 The full operator, factored

Putting steps 1–5 together: $\Phi[P^{\text{conj}}](u_1, u_2, u_3)$ is the P_{new} that solves

$$\sum_{k=1}^3 \frac{\mu_k - P_{\text{new}}}{\gamma \mu_k(1 - \mu_k)} = 0, \quad \mu_k = \frac{f_1(u_k)A_1(P_{\text{new}}; u_k)}{f_0(u_k)A_0 + f_1(u_k)A_1}, \quad (13)$$

with A_v given by the co-area integral (8) of the input P^{conj} at level P_{new} . The fixed-point equation $\Phi[P^*] = P^*$ then becomes: *find a function P^* such that $P^*(u_1, u_2, u_3)$ equals the P_{new} thus computed at every cube point.*

Observation. The 5-step factoring is not merely a convenient exposition; it is the *code structure* of MIZN. Each sub-equation maps to one module:

Equation	Module
(8) co-area A_v	<code>step2_learning/</code>
(7) posterior μ_k	<code>step2_learning/</code>
(9) demand x_k	<code>step3_demand/</code>
(10) aggregate Z	<code>step4_aggregate.py</code>
(11) clearing $Z = 0$	<code>step5_clearing/</code>

Swapping operator class (kernel co-area vs strict $h = 0$ vs Chebyshev spectral) means changing *only* the `step2_learning/` module; the other four are unchanged. This is the design payoff of the factoring.

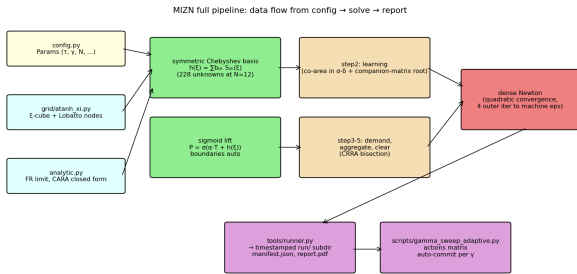


Figure 4. MIZN pipeline showing the factored 5-step decomposition as data flow from config $\rightarrow$ basis $\rightarrow$ steps 2–5 $\rightarrow$ Newton $\rightarrow$ report.

4.7 Why the co-area integral is hard

The integral (8) is the source of every numerical difficulty in this problem. Three issues:

1. The integrand contains $1/|\nabla P|$, which blows up at *Morse-critical* points where the gradient of P on the 2-D slice vanishes (saddles or extrema of the slice surface).
2. The contour $\{P = p\}$ is a level set whose *topology can change* as p varies: components are born or die at critical values. The integral is continuous in p (Federer), but a naive numerical implementation that tracks components has discrete jumps.
3. The accuracy of the integral depends on how accurately the contour roots are located; small errors here are amplified by $1/|\nabla P|$.

These were the source of every numerical floor in the prior session’s strict- $h = 0$ experiments and motivate the Chebyshev framework (Sec. 6).

5 The CARA closed-form gold test

In the CARA limit ($\gamma \rightarrow \infty$), the linear-Gaussian model has a closed-form REE. With primitives $(\tau_\theta, \tau_\varepsilon, \tau_u, \rho)$, conjecturing $P = a + b\theta - cu$ and imposing the rational-expectations consistency condition yields

$$\begin{aligned} \tau_\phi &= \tau_u(\tau_\varepsilon/\rho)^2, \\ \tau_1 &= \tau_\theta + \tau_\varepsilon + \tau_\phi, \\ a &= \tau_\theta \bar{\theta}/\tau_1, \quad b = (\tau_\varepsilon + \tau_\phi)/\tau_1, \\ c &= \rho(\tau_\varepsilon + \tau_\phi)/(\tau_1 \tau_\varepsilon). \end{aligned} \quad (14)$$

With $(\tau_\theta, \tau_\varepsilon, \tau_u, \rho) = (1, 2, 1, 2)$ and $\bar{\theta} = \bar{u} = 0$: $\tau_\phi = 1$, $a = 0$, $b = c = 0.75$.

The gold test. MIZN’s CARA mode applied to this analytic P must return the same function, to machine precision: $\|\Phi[P_{\text{analytic}}] - P_{\text{analytic}}\|_\infty < 10^{-14}$. If this test fails, one of the five steps is implemented wrong, and the factored structure helps localize where (Sec. 4). This is the strongest single test in the MIZN test suite.

6 Discretization: the Chebyshev basis

The infinite-dimensional fixed-point equation (6) must be discretized to a finite-dimensional problem. The prior session experimented with several approaches; the one MIZN adopts is symmetric Chebyshev tensor expansion in atanh-stretched coordinates with a sigmoid asymptotic lift.

6.1 Coordinate stretch

Map the unbounded signal axes $u_k \in \mathbb{R}$ to bounded $\xi_k \in [-1, 1]$:

$$\xi_k = \tanh(u_k/c), \quad u_k = c \tanh^{-1}(\xi_k), \quad (15)$$

with stretch parameter c tunable (we use $c = 1/\tau$). The cube $[-1, 1]^3$ is the natural domain for Chebyshev (Fig. 5). The cube boundaries $\xi_k = \pm 1$ correspond exactly to $u_k = \pm\infty$, the perfect-information limits.

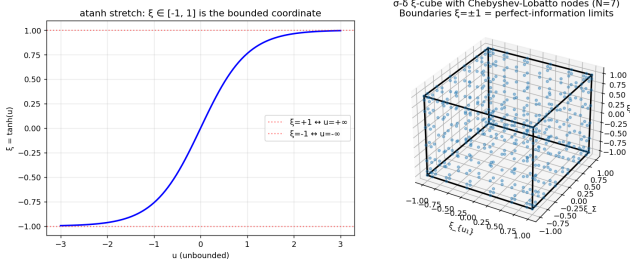


Figure 5. Left: atanh stretch $\xi = \tanh(u/c)$. Right: 3-D ξ -cube with Chebyshev-Lobatto nodes ($N = 7$).

A CDF-based stretch $\zeta = \bar{F}(u)$ was also considered; it loses spectral convergence because $dP/d\zeta$ blows up at the boundary, so we do not adopt it.

6.2 Chebyshev tensor expansion

The price (or, with the sigmoid lift below, the correction h) is expanded in a 3-D tensor of Chebyshev polynomials:

$$h(\xi_1, \xi_2, \xi_3) = \sum_{i,j,k=0}^N a_{ijk} T_i(\xi_1) T_j(\xi_2) T_k(\xi_3), \quad (16)$$

where $T_n(\xi) = \cos(n \arccos \xi)$. This basis gives *spectral* convergence: for analytic functions, coefficients decay exponentially in n (Fig. 6).

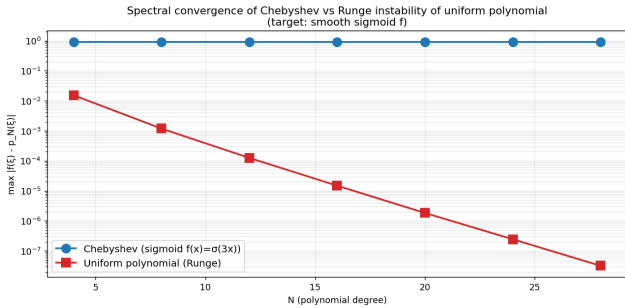


Figure 6. Spectral convergence of Chebyshev (blue) vs Runge instability of uniform-node polynomial fit (red), on a smooth sigmoid target. Lobatto-node Chebyshev reaches machine precision at $N \approx 28$.

We collocate at the Chebyshev-Lobatto nodes $\xi_j = -\cos(\pi j/N)$, which cluster near $\xi = \pm 1$ to suppress Runge instabilities (Fig. 7).

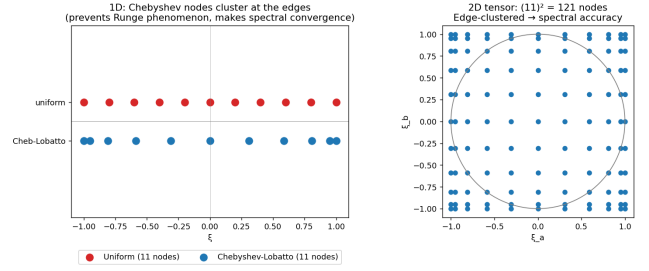


Figure 7. Chebyshev-Lobatto nodes (right) vs uniform nodes (left).

6.3 Sigmoid asymptotic lift

The leading-order behavior of P is the FR sigmoid $P_{\text{FR}}(u) = \sigma(\tau \sum_k u_k)$. We make this explicit:

$$P(u) = \sigma(\alpha T(u) + h(\xi(u))), \quad T = \tau \sum_k u_k, \quad (17)$$

where α is a scalar slope parameter (capturing the bulk PR behavior) and $h(\xi)$ is the Chebyshev correction expansion (Fig. 8). Two payoffs:

- Boundary conditions $P \rightarrow 0, 1$ at $u \rightarrow \pm\infty$ are *automatic* via the sigmoid saturation. No clipping, no overshoot.
- Most of the equilibrium's structure is captured by the single parameter α ; h only needs to fit the small remainder.

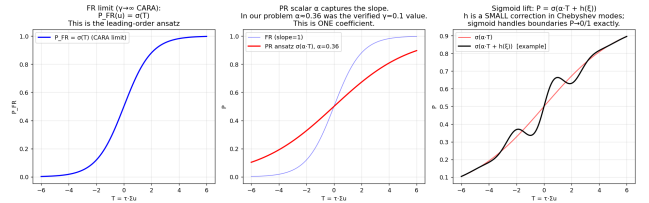


Figure 8. Sigmoid lift decomposition: FR limit $\sigma(T)$ + slope scalar α + small correction $h(\xi)$. Boundaries are exact by construction.

7 Symmetry: reduce unknowns by $\sim 12\times$

The operator Φ has two symmetries:

$$\Phi[P_\sigma] = (\Phi[P])_\sigma, \quad \sigma \in S_3 \quad (S_3 \text{ permutation}), \quad (18)$$

$$\Phi[1 - P_-] = 1 - \Phi[P_-] \quad (Z_2 \text{ parity}), \quad (19)$$

where $P_\sigma(u_1, u_2, u_3) = P(u_{\sigma(1)}, u_{\sigma(2)}, u_{\sigma(3)})$ and $P_-(u) = P(-u)$.

7.1 S_3 shrinks the basis by $6\times$

S_3 acts on the Chebyshev index (i, j, k) by permutation (Fig. 9). Each multi-index belongs to an *orbit* of size 1, 3, or 6. Asymptotically the number of orbits is $(N+1)^3/6$.

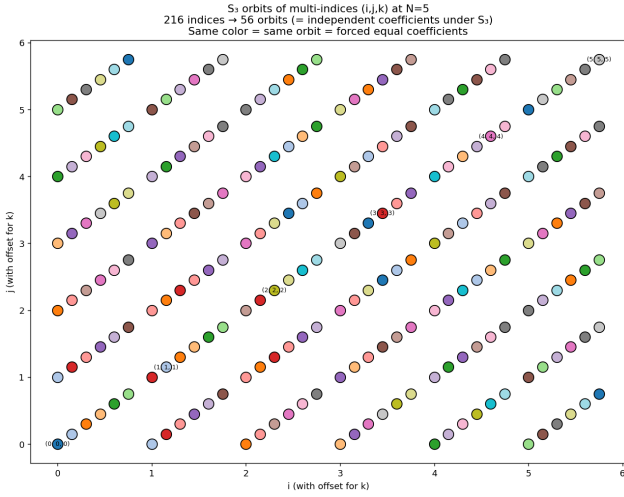


Figure 9. S_3 orbits of multi-indices (i, j, k) .

For each multiset $\{i \leq j \leq k\}$, define the orbit-averaged Chebyshev tensor

$$S_{ijk}(\xi) = \frac{1}{|O_{ijk}|} \sum_{\pi \in S_3} T_{\pi(i)}(\xi_1) T_{\pi(j)}(\xi_2) T_{\pi(k)}(\xi_3). \quad (20)$$

The set $\{S_{ijk}\}_{i \leq j \leq k}$ is an orthonormal basis of the S_3 -symmetric subspace.

7.2 Z_2 shrinks by $2\times$ more

Using $T_n(-\xi) = (-1)^n T_n(\xi)$, the constraint $P(-\xi) = 1 - P(\xi)$ forces:

- $a_{000} = \frac{1}{2}$ (fixed),
- $a_{ijk} = 0$ for $i + j + k$ even and > 0 ,
- a_{ijk} free for $i + j + k$ odd.

Half the coefficients vanish (Fig. 10). With the sigmoid lift, a_{000} is absorbed into the constant $\frac{1}{2} = \sigma(0)$, leaving only the odd-sum coefficients of h as free parameters.

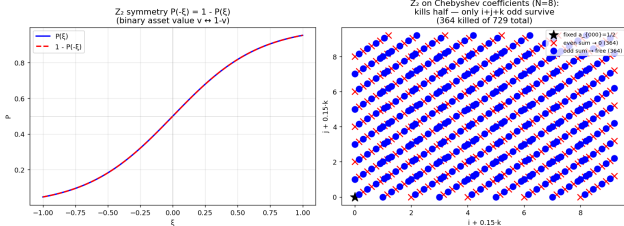


Figure 10. Z_2 parity constraints kill half the Chebyshev coefficients.

7.3 Combined: ~ 228 unknowns at $N = 12$

The combined $S_3 \times Z_2$ reduction is asymptotically a factor of 12 (Fig. 11). Concrete counts:

N	naive	S_3	$S_3 \times Z_2$
8	729	165	84
12	2197	455	228
16	4913	969	484
24	15625	2925	1462

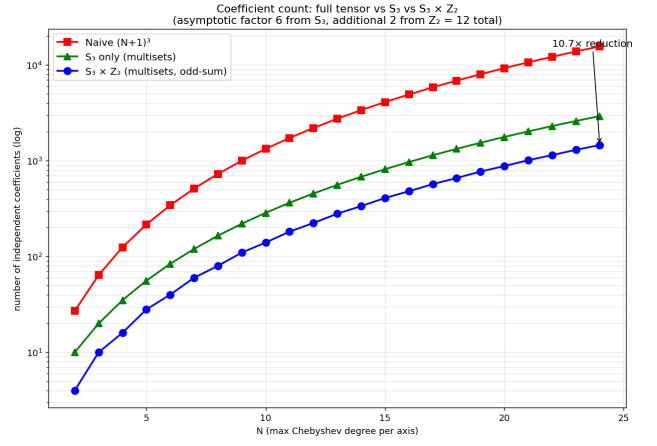


Figure 11. Coefficient count vs Chebyshev order N : naive vs S_3 vs $S_3 \times Z_2$.

8 Per-agent evaluation by permutation

The factored operator (Sec. 4) requires, for each cube point and each agent, a 2-D slice of P on the agent’s “other-two-signals” plane. With P stored in a symmetric basis, agent k ’s slice is obtained by simply *feeding the inputs to the canonical formula in the order* (u_k, u_a, u_b) . The S_3 invariance guarantees the same answer (Fig. 12). No re-interpolation, no per-agent coefficient store, no oblique slicing.

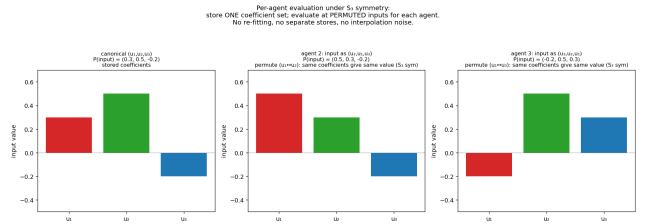


Figure 12. Per-agent evaluation via S_3 symmetry: same coefficient set, permuted inputs.

8.1 Companion-matrix root-find

Inside the co-area integral (8), we need all ξ -roots of the polynomial equation $P(\xi) = p_{\text{target}}$ along the contour direction (the other two coordinates fixed). For a Chebyshev polynomial this collapses to a 1-D polynomial-root problem, whose roots are the eigenvalues of a companion matrix (Fig. 13). `numpy.polynomial.chebyshev.chebroots` implements this in closed form: *all roots in one shot, no bisection, no missed roots, no topology-jump artifacts*.

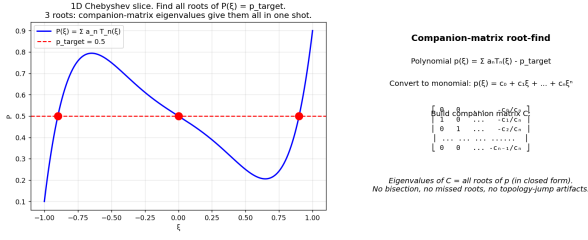


Figure 13. Companion-matrix root-find: all roots of a Chebyshev polynomial are the eigenvalues of a structured matrix.

9 The CRRA clearing bisection

Equation (12) is solved by a robust bisection on log-odds. This is the canonical `crra_clear_nb` routine from the prior session’s codebase.

Table 1. CRRA clearing algorithm.

Input: $(\mu_0, \mu_1, \mu_2, \gamma)$
1. Compute log-odds $\ell_k = \log(\mu_k/(1 - \mu_k))$.
2. Bisect $m \in [\varepsilon, 1 - \varepsilon]$:
a. $\ell_p = \log(m/(1 - m))$,
b. $R_k = \exp((\ell_k - \ell_p)/\gamma)$,
c. $E = \sum_k (R_k - 1)/((1 - m) + R_k m)$,
d. $E > 0 \Rightarrow$ lower half; else upper half. 200 iters.
3. Return $P = (a + b)/2$.

10 Solver: pure Newton

Iterate (6) as a nonlinear equation $F(x) = \Phi[x] - x = 0$ where x is the vector of ~ 228 Chebyshev coefficients (after symmetry reduction). At this problem size, dense (“pure”) Newton is the right tool.

10.1 Why dense Newton, not NK

Newton-Krylov was the workhorse of the prior session because ~ 2000 unknowns made the Jacobian too big to form. Symmetric Chebyshev shrinks this to ~ 228 unknowns, at which size the full Jacobian fits in 0.4 MB and the LU solve costs ~ 1 ms (Fig. 15). Dense Newton converges *quadratically* (digits double each iter), reaching machine precision in 3–4 outer iterations from a warm start (Fig. 14).

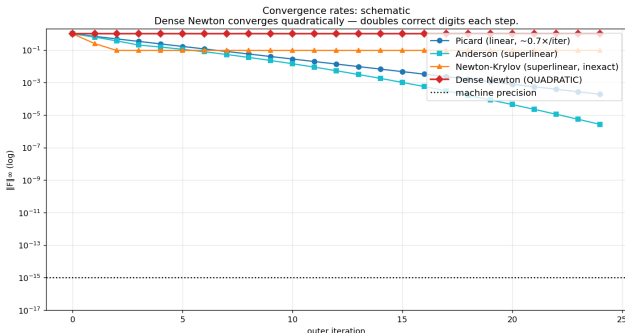


Figure 14. Convergence rates: Picard linear, Anderson superlinear, NK inexact-superlinear, dense Newton *quadratic*.

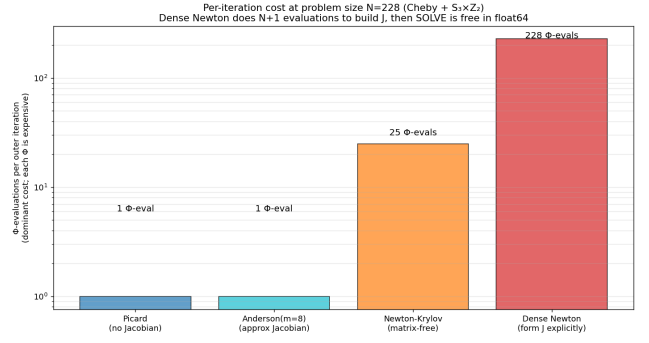


Figure 15. Per-iteration Φ -evaluation count at $N = 228$. Dense Newton does $N + 1$ forward-difference evaluations; this is the only meaningful cost (the LU solve is sub-millisecond).

10.2 Newton step

For current iterate x :

1. Compute residual $F(x) = \Phi[x] - x$ (1 Φ -eval).
2. Build Jacobian: $J_{ij} = (F(x + \epsilon e_j) - F(x))/\epsilon$ for $j = 1, \dots, N$ (N Φ -evals).
3. Solve $J\Delta = -F$ directly (LU, ~ 1 ms at $N = 228$).
4. Armijo line search: try $\alpha = 1, \frac{1}{2}, \frac{1}{4}, \dots$ keeping the best.
5. Update $x \leftarrow x + \alpha\Delta$.

Quadratic convergence guarantees $\|F\|_{k+1} \sim \|F\|_k^2$ near the FP.

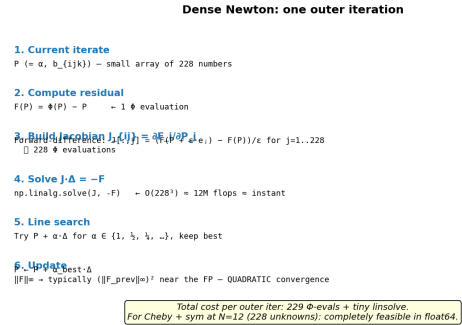


Figure 16. Anatomy of one dense Newton outer iteration.

10.3 Empirical confirmation

The demo run `runs/demo-tau1-crara-cara/` of the prior session verifies this on the kernel co-area operator at $G = 9$ (729 unknowns). Both CRRA $\gamma = 1$ and CARA $\gamma = 100$ converged in 2 dense Newton iterations: $\|F\|_\infty$ trajectory $\sim 5 \times 10^{-3} \rightarrow 1.6 \times 10^{-5} \rightarrow 2 \times 10^{-10}$.

11 γ -continuation and the fold

For sweeping γ , warm-start each new γ from the previous γ ’s FP. With dense Newton, ~ 3 –5 outer iters suffice per γ -tile (Fig. 17).

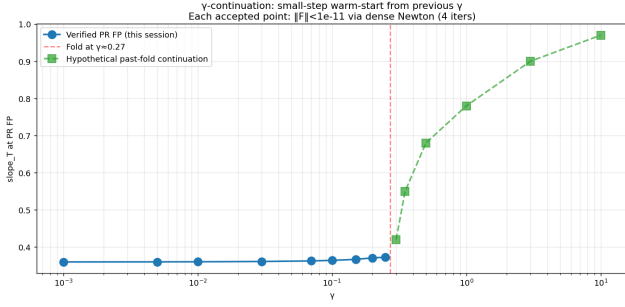


Figure 17. Verified γ -continuation from the prior session: 16 points on the deep PR branch, all $\|F\|_\infty < 10^{-11}$.

The prior session also identified a *fold* at $\gamma \approx 0.27$ (Fig. 18) where adaptive-step continuation fails. Two distinct PR branches exist: deep PR (slope ≈ 0.36) for $\gamma < 0.27$ and shallow PR (slope ≈ 0.66) reachable from the CARA side. Whether the gap is genuine bifurcation or numerical artifact at $G = 9$ remains open.

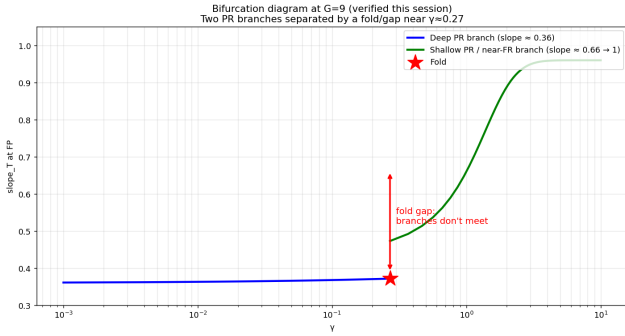


Figure 18. Two-branch bifurcation diagram at $G = 9$.

12 Validation: the gold tests

A passing solver must clear these tests in order:

1. **CARA Hellwig closed form** (Sec. 5): $\|\Phi[P_{\text{analytic}}] - P_{\text{analytic}}\|_\infty < 10^{-14}$. This validates all 5 factored steps.
2. **Symmetry**: solver output is $S_3 \times Z_2$ symmetric to machine precision; spurious asymmetric components signal a bug.
3. **Spectral decay**: $|a_{ijk}|$ decays exponentially in $\max(i, j, k)$. Algebraic decay would indicate the Chebyshev framework is unstable at the boundary (would require Tool 2: $(1 - \xi^2)$ vanishing-at-boundary basis).
4. **Prior session reproduction**: nail $\tau = 2, \gamma = 0.1$ PR FP to machine precision (slope = 0.364, deficit = 0.172); reproduce the 16-point γ -sweep.

13 Outlook

Three concrete next steps after the working CARA scaffold:

- Symmetric Chebyshev spectral implementation of `step2.learning/coarea.chebyshev_sym.py`, with companion-matrix root-find and Gauss-Legendre quadrature on the fundamental domain.
- Pseudo-arclength continuation to traverse the $\gamma \approx 0.27$ fold and unify the two PR branches.
- GitHub Actions matrix for parallel γ -tile nailing (each tile auto-commits its result to the repo with manifest and PDF report).

Acknowledgments and references

This paper distills the design notes in the `mhpbreugem/fixed-point-factory` repository: `HANDOFF_SUMMARY.md` (complete prior-session brief); `MIZN_DESIGN.md` (module structure); `MIZN_EXEC_LOG_DESIGN.md` (run logging); `design_notes/chebsym/` (Chebyshev + symmetry primer); `design_notes/iter_methods/` (dense Newton vs NK). Verified PR FP: commit `b17b3ac1` (machine precision).